

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I MITHRA DEVIS - 192524203 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mithra Devi

Annexure A

Reverse Engineering

1. System Decomposition & Analysis

Analyze an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

QUESTION 1 – SYSTEM DECOMPOSITION & ANALYSIS :

1. Aim

To reverse engineer an existing distributed AI-based fraud detection system and identify its major components, data flow, processing techniques, and underlying algorithms.

2. Problem Understanding

Financial institutions process millions of transactions every day. Detecting fraudulent transactions using a traditional single-machine system can be slow when the dataset becomes very large.

The analyzed system uses **PySpark** to distribute transaction processing across multiple machines. The processed data is then given to a machine-learning model for fraud prediction.

3. System Components

The major components identified during reverse engineering are:

1. Data Source

- Contains transaction records.
- Includes amount, transaction type, location, time, and customer information.

2. Data Ingestion Module

- Reads CSV, JSON, or database data.
- Transfers raw data into the Spark environment.

3. Preprocessing Module

- Removes missing and duplicate records.
- Converts categorical values into numerical form.
- Performs feature transformation.

4. RDD/Data Processing Module

- Uses Spark RDD operations such as `map()`, `filter()`, and `reduceByKey()`.
- Distributes processing across worker nodes.

5. Machine Learning Module

- Uses a classification algorithm.
- Possible algorithms include Logistic Regression, Decision Tree, Random Forest, or Gradient-Boosted Trees.

6. Prediction Module

- Receives processed transaction features.
- Predicts whether a transaction is legitimate or fraudulent.

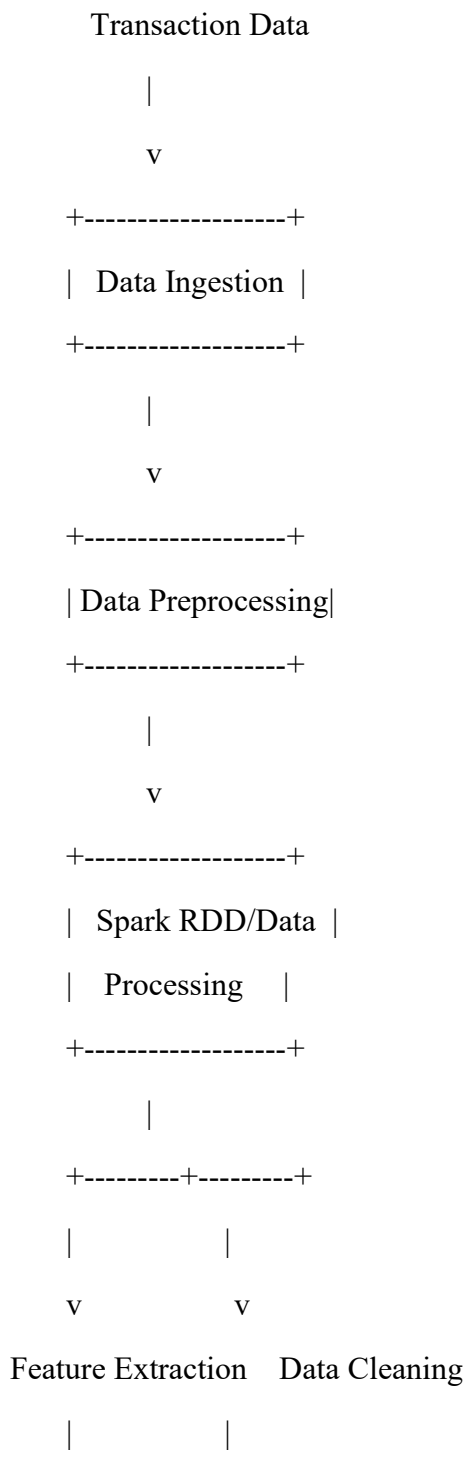
7. **Decision Module**

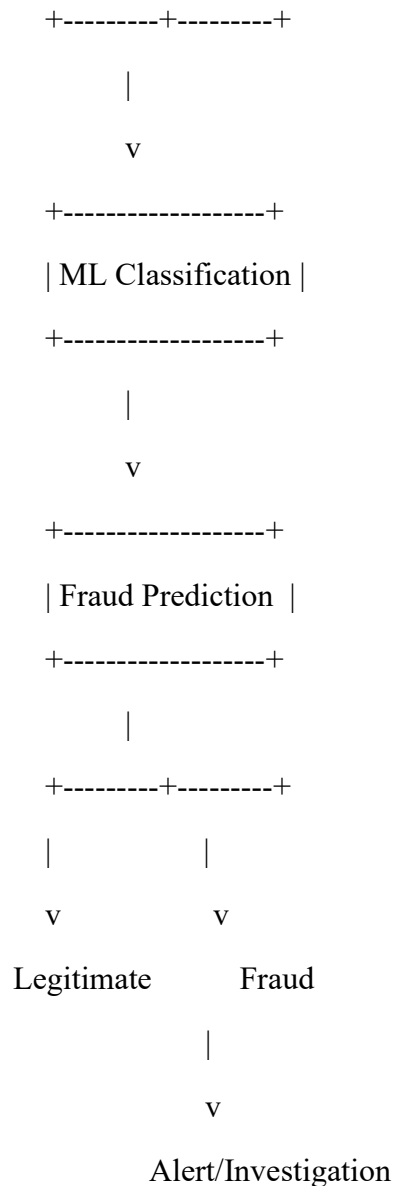
- Applies a threshold or classification result.
- Suspicious transactions are flagged for further investigation.

8. **Output/Monitoring Module**

- Stores prediction results.
- Provides fraud statistics and system performance information.

4. **Architecture**





5. Data Flow

The transaction data first enters the system through the ingestion layer. Spark distributes the data across worker nodes. Preprocessing operations clean and transform the data.

The resulting features are passed to the machine-learning model. The model produces a prediction, and the decision module identifies suspicious transactions.

6. Underlying Algorithms

The reverse-engineered system can use:

- RDD map() for transformation.
- RDD filter() for selecting relevant transactions.
- reduceByKey() for aggregation.
- Logistic Regression or Random Forest for classification.
- Feature scaling for numerical attributes.

- Classification threshold for final decision-making.

7. Analysis

The system is scalable because Spark divides large datasets into partitions and processes them in parallel. RDD transformations are generally evaluated lazily, allowing Spark to optimize execution.

The major limitation is that machine-learning accuracy depends strongly on the quality and balance of the transaction dataset.

8. Conclusion

The reverse-engineered system consists of data ingestion, preprocessing, distributed processing, feature extraction, machine learning, prediction, and decision-making components. PySpark improves scalability by distributing data processing across multiple worker nodes.

Python Code :

```
transactions = [  
    ("T101", 500, "Normal"),  
    ("T102", 12000, "Fraud"),  
    ("T103", 800, "Normal"),  
    ("T104", 15000, "Fraud"),  
    ("T105", 700, "Normal")  
]  
  
print("SYSTEM DECOMPOSITION")  
  
print("-----")  
  
print("1. Data Source")  
  
print("2. Data Preprocessing")  
  
print("3. Feature Processing")  
  
print("4. Machine Learning")  
  
print("5. Decision Making")  
  
print("\nTransaction Analysis:")  
  
for t in transactions:  
    if t[1] > 10000:  
        prediction = "Fraud"  
    else:  
        prediction = "Normal"
```

```
print(t[0], "Amount:", t[1], "Actual:", t[2],  
      "Prediction:", prediction)
```

OUTPUT :

SYSTEM DECOMPOSITION

1. Data Source
2. Data Preprocessing
3. Feature Processing
4. Machine Learning
5. Decision Making

Transaction Analysis:

T101 Amount: 500 Actual: Normal Prediction: Normal
T102 Amount: 12000 Actual: Fraud Prediction: Fraud
T103 Amount: 800 Actual: Normal Prediction: Normal
T104 Amount: 15000 Actual: Fraud Prediction: Fraud
T105 Amount: 700 Actual: Normal Prediction: Normal

QUESTION 2 – ARCHITECTURE RECONSTRUCTION :

1. Aim

To reconstruct the architecture of a black-box distributed AI system and explain how its internal modules communicate with each other.

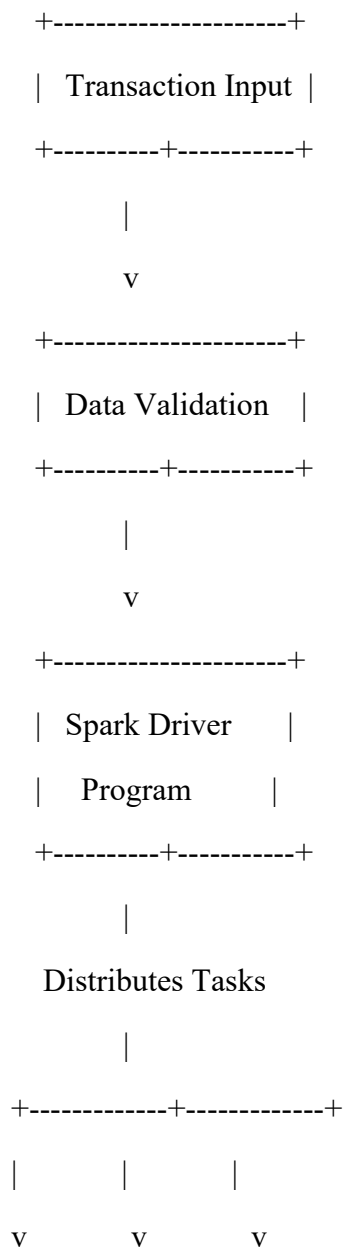
2. Black-Box System

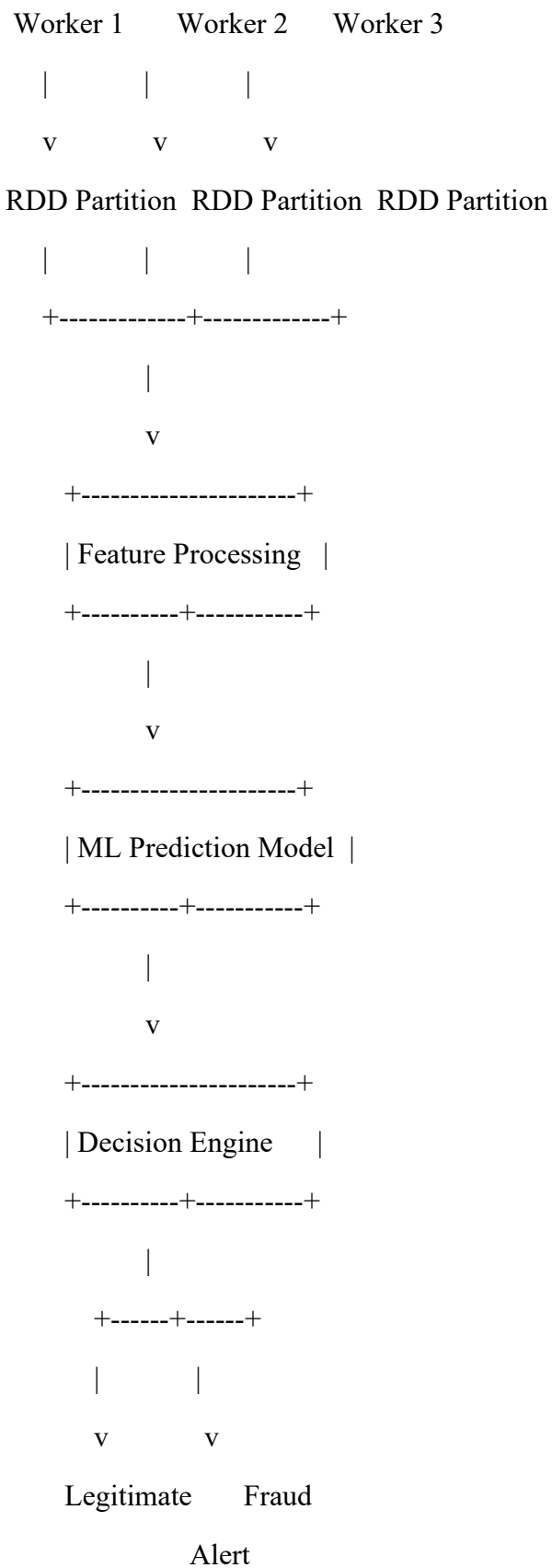
The black-box system receives transaction information and produces one of two outputs:

- **Legitimate Transaction**
- **Fraudulent Transaction**

Since the internal implementation is initially unknown, input-output behavior is analyzed to reconstruct the architecture.

3. Reconstructed Architecture





4. Module Interaction

Module 1 – Input Module

The input module accepts transaction records from files, databases, or streaming sources.

Module 2 – Driver Program

The Spark driver controls the distributed computation. It creates the Spark context/session and sends tasks to worker nodes.

Module 3 – Worker Nodes

Worker nodes process individual partitions of the transaction dataset.

Module 4 – RDD Processing

RDD transformations are applied to distributed transaction data.

Example:

Raw Transactions

|

v

filter()

|

v

map()

|

v

reduceByKey()

Module 5 – Feature Processing

Important transaction attributes are converted into a suitable representation for the ML model.

Module 6 – Machine Learning

The trained model receives the processed features and calculates a fraud prediction.

Module 7 – Decision Engine

The prediction is converted into a final business decision.

5. Communication Between Modules

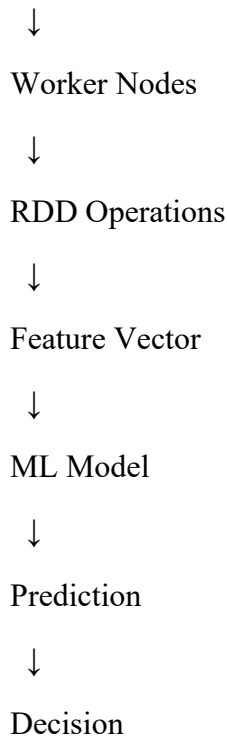
Input

↓

Validation

↓

Spark Driver



The Spark driver communicates with executors running on worker nodes. The worker nodes process their assigned partitions and return required results.

6. Reconstructed System Characteristics

- Distributed processing
- Parallel execution
- Fault tolerance
- Lazy evaluation
- Machine-learning-based decision making
- Scalable data processing

7. Conclusion

By observing the inputs, outputs, processing behavior, and performance characteristics, the black-box system can be reconstructed as a distributed AI architecture consisting of a Spark driver, worker nodes, RDD processing, feature engineering, machine learning, and a decision engine.

PYTHON CODE :

```
def data_input():  
    return [500, 12000, 800, 15000, 700]  
  
def preprocessing(data):  
    return [x for x in data if x > 0]  
  
def prediction(data):
```

```

result = []

for amount in data:

    if amount > 10000:

        result.append("Fraud")

    else:

        result.append("Normal")

return result

print("RECONSTRUCTED AI ARCHITECTURE")

print("-----")

data = data_input()

print("Input Data:", data)

clean_data = preprocessing(data)

print("Preprocessed Data:", clean_data)

result = prediction(clean_data)

print("Predictions:", result)

print("\nModules:")

print("Input -> Preprocessing -> Prediction -> Decision")

```

OUTPUT :

RECONSTRUCTED AI ARCHITECTURE

Input Data: [500, 12000, 800, 15000, 700]

Preprocessed Data: [500, 12000, 800, 15000, 700]

Predictions: ['Normal', 'Fraud', 'Normal', 'Fraud', 'Normal']

Modules:

Input -> Preprocessing -> Prediction -> Decision

QUESTION 3 – ALGORITHM IDENTIFICATION :

1. Aim

To reverse engineer a machine-learning application and identify its data-processing operations, learning algorithm, and decision-making logic.

2. Identified Processing Pipeline

The reverse-engineered pipeline is:

Raw Dataset

|

v

Data Cleaning

|

v

Missing Value Handling

|

v

Feature Transformation

|

v

RDD Processing

|

v

Feature Vector

|

v

Machine Learning

|

v

Prediction

|

v

Decision

3. RDD Operations Identified

map()

The map() operation transforms every transaction record.

```
rdd.map(lambda x: (x[0], float(x[1])))
```

It can be used to convert transaction values into the required format.

filter()

The filter() operation removes unwanted records.

```
rdd.filter(lambda x: x[1] > 0)
```

It keeps only transactions satisfying the specified condition.

reduceByKey()

The reduceByKey() operation combines values belonging to the same key.

```
rdd.reduceByKey(lambda a, b: a + b)
```

It can be used to calculate total transaction amounts or transaction counts.

4. Machine Learning Algorithm

A suitable reverse-engineered algorithm is **Random Forest Classification**.

Random Forest creates multiple decision trees and combines their predictions to obtain a final classification.

The classification process can be represented as:

Transaction Features

|

v

Decision Tree 1

|

v

Decision Tree 2

|

v

Decision Tree 3

|

v

...

|

v

Majority Voting

|

v

Fraud / Legitimate

5. Decision-Making Logic

The system uses the model output to classify transactions.

For example:

If fraud probability $\geq$ threshold

|

v

FRAUD

Otherwise

|

v

LEGITIMATE

A threshold such as 0.5 can be used, although the actual threshold should be selected according to the application's validation results and business requirements.

6. Identified Algorithms

Stage	Algorithm/Technique
Data cleaning	Filtering
Transformation	map()
Selection	filter()
Aggregation	reduceByKey()
Feature processing	Feature transformation
Classification	Random Forest
Decision	Probability/threshold-based classification

7. Performance Analysis

Random Forest can provide strong classification performance and is suitable for nonlinear relationships between transaction features.

PySpark allows the data-processing stage to scale across multiple nodes. However, model training and prediction performance depend on dataset size, number of trees, feature count, cluster resources, and class imbalance.

8. Conclusion

The reverse-engineered application uses distributed data processing followed by machine-learning classification. RDD transformations prepare the data, while Random Forest performs fraud classification. The final decision is generated using the model prediction and classification threshold.

PYTHON CODE :

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("AlgorithmIdentification") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext

data = [
    ("Normal", 500),
    ("Fraud", 12000),
    ("Normal", 800),
    ("Fraud", 15000),
    ("Normal", 700)
]

rdd = sc.parallelize(data)

# map operation
mapped = rdd.map(lambda x: (x[0], x[1] * 2))

# filter operation
filtered = rdd.filter(lambda x: x[1] > 1000)

# reduceByKey operation
total = rdd.reduceByKey(lambda a, b: a + b)

print("ALGORITHM IDENTIFICATION")
```

```
print("-----")
print("\nOriginal Data:")
print(rdd.collect())
print("\nAfter map():")
print(mapped.collect())
print("\nAfter filter():")
print(filtered.collect())
print("\nAfter reduceByKey():")
print(total.collect())
spark.stop()
```

OUTPUT :

ALGORITHM IDENTIFICATION

Original Data:

```
[('Normal', 500), ('Fraud', 12000),
 ('Normal', 800), ('Fraud', 15000),
 ('Normal', 700)]
```

After map():

```
[('Normal', 1000), ('Fraud', 24000),
 ('Normal', 1600), ('Fraud', 30000),
 ('Normal', 1400)]
```

After filter():

```
[('Fraud', 12000), ('Fraud', 15000)]
```

After reduceByKey():

```
[('Normal', 2000), ('Fraud', 27000)]
```

QUESTION 4 – DATA FLOW & PROCESS MAPPING :

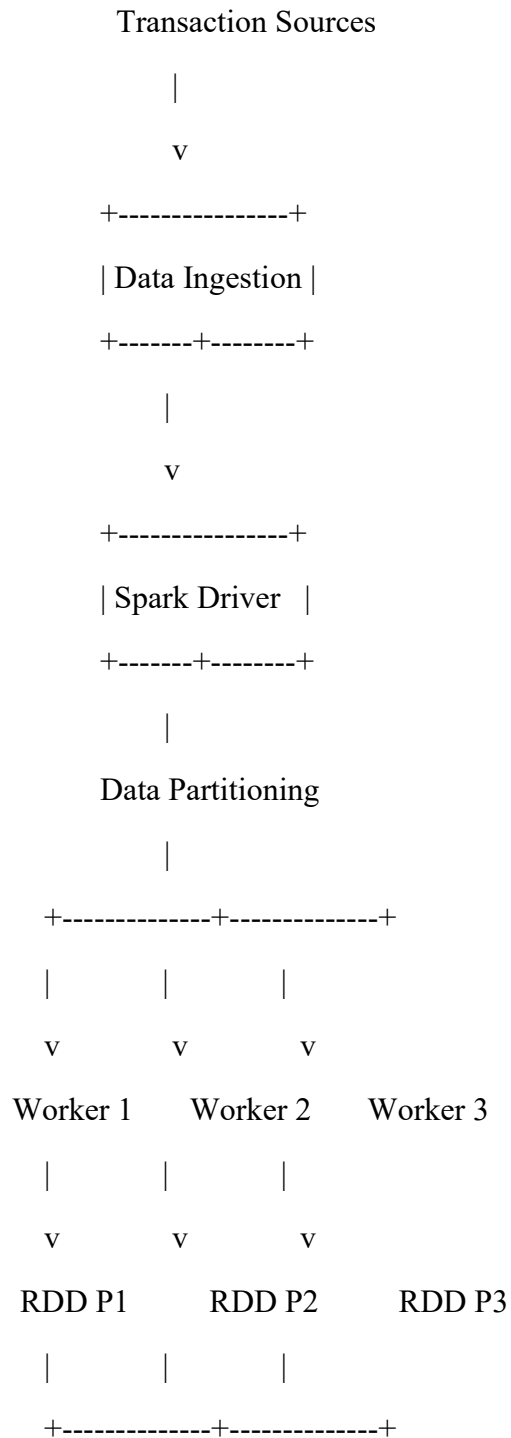
1. Aim

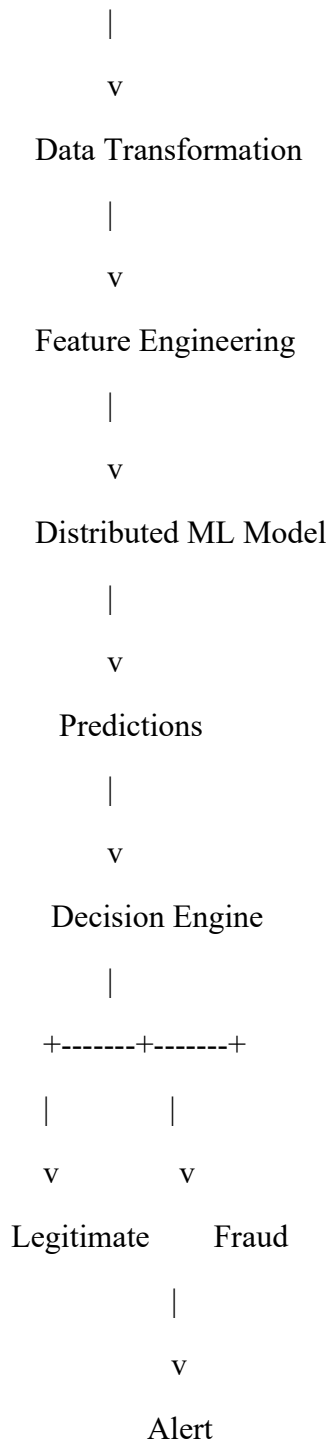
To examine a distributed AI application and map its complete data flow, processing pipeline, and communication between distributed components.

2. Distributed System

The selected system is a **PySpark-based distributed fraud detection application**.

3. Data Flow Diagram





4. Processing Pipeline

Step 1 – Data Collection

Transaction data is collected from financial transaction sources.

Step 2 – Data Ingestion

The Spark application loads the dataset into the distributed processing environment.

Step 3 – Partitioning

Spark divides the dataset into partitions.

For example:

10,000,000 records

```
|
+---- Worker 1 → 2,500,000
+---- Worker 2 → 2,500,000
+---- Worker 3 → 2,500,000
+---- Worker 4 → 2,500,000
```

Step 4 – Data Processing

Each worker processes its assigned partition.

RDD operations such as:

- map()
- filter()
- flatMap()
- reduceByKey()

can be used.

Step 5 – Feature Engineering

Relevant transaction attributes are converted into machine-learning features.

Step 6 – Model Prediction

The trained AI model predicts whether each transaction is suspicious.

Step 7 – Decision Making

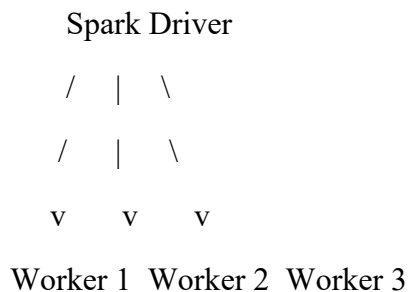
The decision engine categorizes transactions.

Step 8 – Output

The final results are stored or sent to an alerting system.

5. Communication

The Spark driver coordinates the workers.



```

\   |   /
\   |   /
v   v   v

```

Results

```

|
v

```

Driver/Output

The driver distributes tasks, while worker nodes execute the operations on their assigned partitions.

6. Distributed AI Advantages

- Handles large datasets.
- Provides parallel processing.
- Reduces processing time.
- Supports horizontal scalability.
- Provides fault tolerance through Spark's execution model.
- Can integrate machine-learning pipelines.

7. Conclusion

The reverse-engineered data flow shows that the application uses a distributed processing architecture. Data moves from transaction sources to the Spark driver, worker nodes, RDD transformations, feature processing, machine learning, and finally the decision and alerting modules.

PYTHON CODE :

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
```

```
    .appName("DataFlow") \
```

```
    .master("local[*]") \
```

```
    .getOrCreate()
```

```
sc = spark.sparkContext
```

```
transactions = [
```

```
    ("T101", 500),
```

```
    ("T102", 12000),
```

```
    ("T103", 800),
```

```
    ("T104", 15000),
```

```

("T105", 700)

]

print("DATA FLOW PROCESS")
print("-----")
# Step 1: Input
rdd = sc.parallelize(transactions)
print("1. Input Data:", rdd.collect())
# Step 2: Filtering
valid = rdd.filter(lambda x: x[1] > 0)
print("2. Valid Data:", valid.collect())
# Step 3: Feature transformation
features = valid.map(lambda x: (x[0], x[1], x[1] / 1000))
print("3. Features:", features.collect())
# Step 4: Prediction
prediction = features.map(
    lambda x: (x[0], x[1],
               "Fraud" if x[1] > 10000 else "Normal")
)
print("4. Predictions:")
for item in prediction.collect():
    print(item)
# Step 5: Final output
print("\n5. Final Output:")
print("Data -> Processing -> Features -> Prediction -> Decision")
spark.stop()

```

OUTPUT :

DATA FLOW PROCESS

1. Input Data:

[('T101', 500), ('T102', 12000), ('T103', 800),
('T104', 15000), ('T105', 700)]

2. Valid Data:

[('T101', 500), ('T102', 12000), ('T103', 800),
('T104', 15000), ('T105', 700)]

3. Features:

[('T101', 500, 0.5), ('T102', 12000, 12.0),
('T103', 800, 0.8), ('T104', 15000, 15.0),
('T105', 700, 0.7)]

4. Predictions:

('T101', 500, 'Normal')
('T102', 12000, 'Fraud')
('T103', 800, 'Normal')
('T104', 15000, 'Fraud')
('T105', 700, 'Normal')

5. Final Output:

Data -> Processing -> Features -> Prediction -> Decision

QUESTION 5 – IMPROVEMENT & REDESIGN :

1. Aim

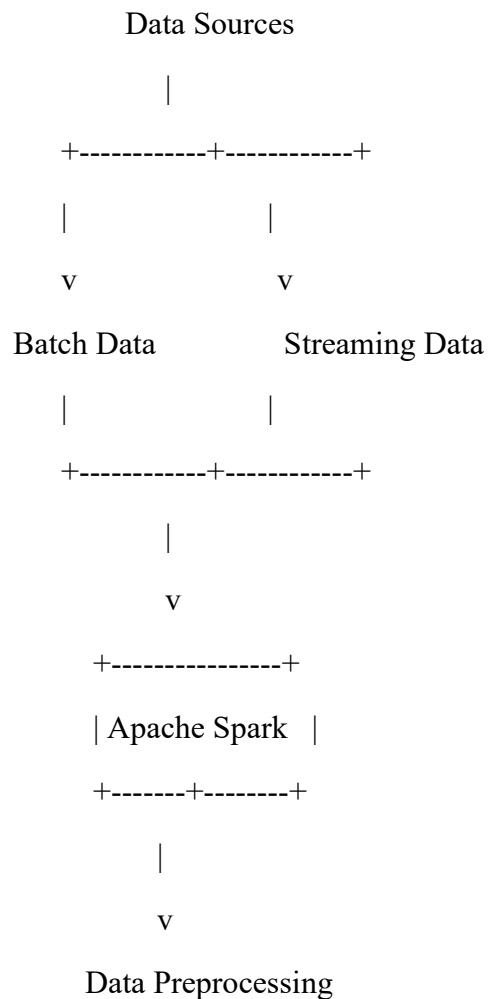
To reverse engineer an intelligent distributed system and propose improvements that increase scalability, efficiency, reliability, and prediction performance.

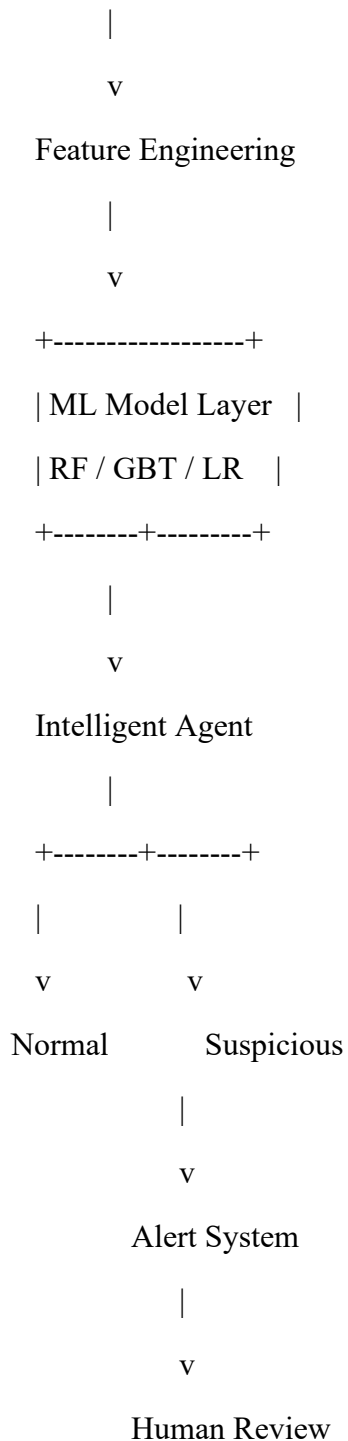
2. Existing System Limitations

The reverse-engineered system may have the following limitations:

1. Processing very large datasets can increase execution time.
2. Repeated RDD transformations can create unnecessary computation.
3. Imbalanced fraud datasets can reduce classification quality.
4. Batch processing may not detect fraud immediately.
5. Poor feature selection can reduce model accuracy.
6. Increasing data volume may require additional computational resources.
7. A single model may not perform equally well for all transaction patterns.

3. Proposed Redesign





4. Proposed Improvements

1. Use DataFrames

Instead of performing all operations using low-level RDDs, Spark DataFrames can be used for structured data processing.

Advantages:

- Better optimization.
- Easier data manipulation.

- Improved readability.
- Efficient execution using Spark's query optimizer.

2. Use Caching

Frequently accessed datasets can be cached.

`data.cache()`

This avoids repeatedly reading and calculating the same data.

3. Improve Feature Engineering

Additional features can be created from:

- Transaction frequency.
- Average transaction amount.
- Time between transactions.
- Location changes.
- Unusual spending patterns.

These features can improve fraud detection.

4. Handle Class Imbalance

Fraud datasets usually contain many more legitimate transactions than fraudulent transactions.

Possible approaches include:

- Class weighting.
- Oversampling.
- Undersampling.
- Appropriate evaluation metrics.

Accuracy alone should not be used to evaluate the system.

5. Use Streaming Processing

For real-time fraud detection, Spark Structured Streaming can be incorporated.

Live Transaction

|

v

Streaming Layer

|

v

Real-Time Feature Processing

|

v

ML Prediction

|

v

Immediate Fraud Alert

This reduces the delay between transaction occurrence and fraud detection.

6. Intelligent Agent Integration

An intelligent monitoring agent can observe predictions and automatically decide whether to:

- Allow the transaction.
- Flag the transaction.
- Request additional verification.
- Send an alert to an investigator.

The agent can use the current transaction state, model prediction, and risk score to select an action.

5. Performance Evaluation

The redesigned system should be evaluated using:

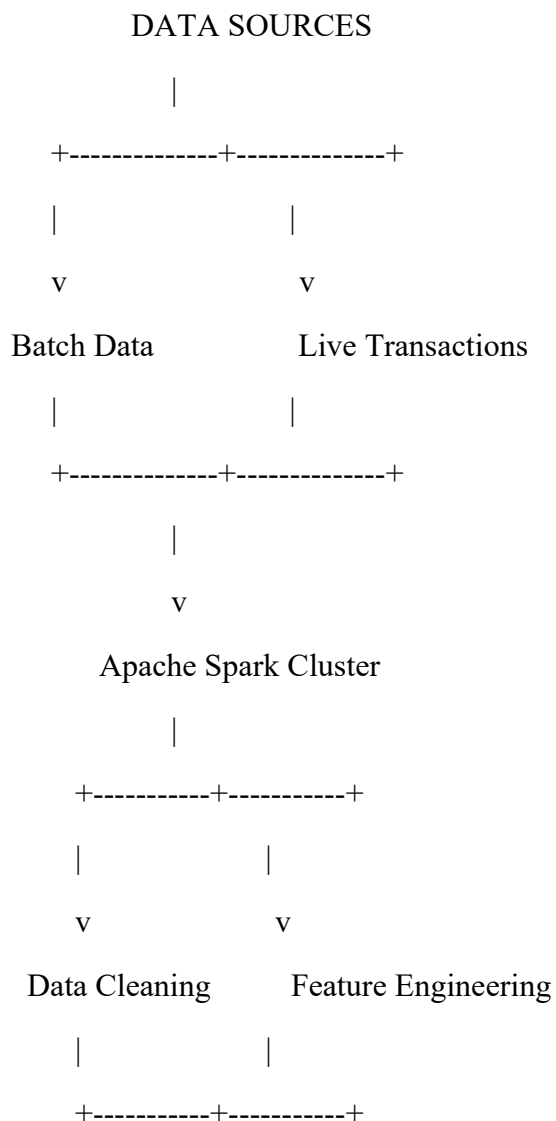
- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Processing time
- Throughput
- Memory utilization
- Scalability

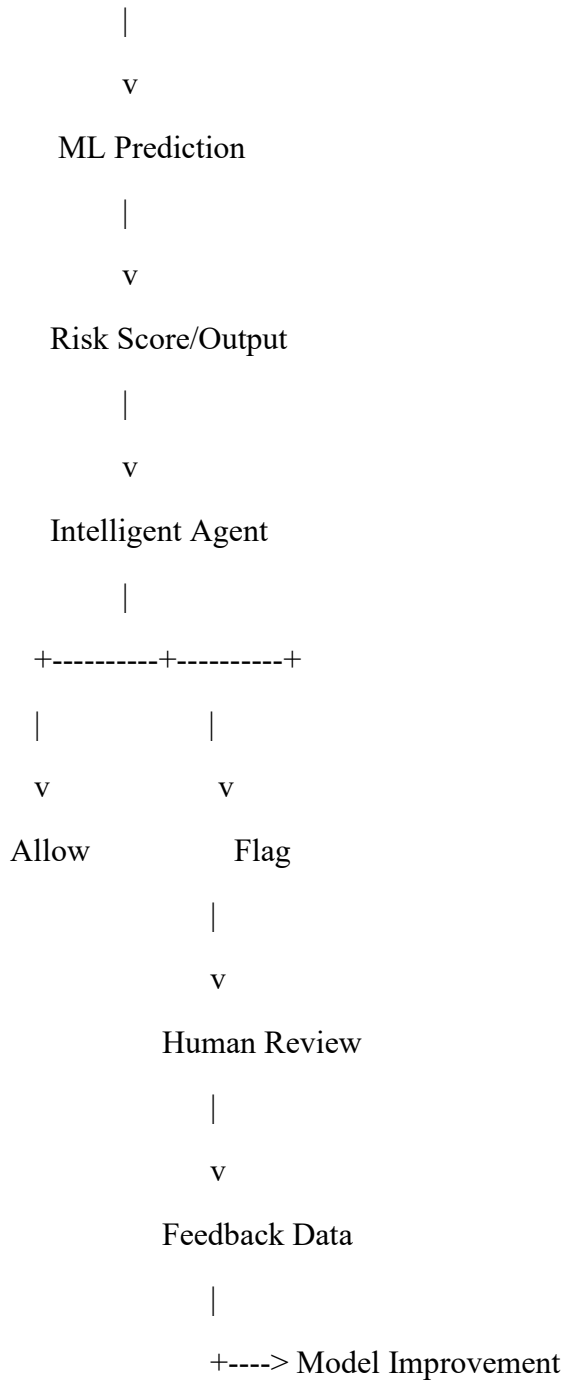
For fraud detection, **precision and recall are particularly important** because both false alarms and missed fraudulent transactions have significant consequences.

6. Expected Improvements

Parameter	Existing System	Redesigned System
Data processing	Distributed	Optimized distributed
Processing type	Mainly batch	Batch + streaming
Data structure	RDD	RDD + DataFrame
Feature engineering	Basic	Advanced
Model	Single model	Multiple model options
Fraud response	Delayed	Near real-time
Scalability	Good	Higher
Monitoring	Basic	Intelligent monitoring
Evaluation	Accuracy	Multiple metrics

7. Final Redesigned Architecture





8. Conclusion

The redesigned system improves the original architecture by combining distributed Spark processing, optimized data structures, advanced feature engineering, real-time processing, machine-learning prediction, and intelligent-agent decision making. The proposed architecture is more scalable, efficient, and suitable for large-scale AI applications.

The redesign also introduces continuous feedback, allowing future model versions to learn from newly identified fraudulent transactions and improve system performance.

PYTHON CODE :

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("ImprovedFraudDetection") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext

data = [
    ("T101", 500),
    ("T102", 12000),
    ("T103", 800),
    ("T104", 15000),
    ("T105", 700),
    ("T106", 25000)
]

rdd = sc.parallelize(data)

# Cache data for faster repeated processing
rdd.cache()

# Improved feature processing
features = rdd.map(
    lambda x: (
        x[0],
        x[1],
        "High Risk" if x[1] > 10000 else "Low Risk"
    )
)

print("IMPROVED AI SYSTEM")

print("-----")

print("\nOriginal Transactions:")

for x in rdd.collect():
```

```

    print(x)

print("\nRisk Analysis:")
for x in features.collect():
    print(x)

# Count high-risk transactions
high_risk = features.filter(
    lambda x: x[2] == "High Risk"
).count()

low_risk = features.filter(
    lambda x: x[2] == "Low Risk"
).count()

print("\nPerformance Summary")
print("-----")
print("High Risk Transactions:", high_risk)
print("Low Risk Transactions:", low_risk)
print("\nImprovements:")
print("1. Distributed Processing")
print("2. RDD Caching")
print("3. Feature Engineering")
print("4. Risk-Based Decision Making")
print("5. Scalable Architecture")

spark.stop()

```

OUTPUT :

IMPROVED AI SYSTEM

Original Transactions:

('T101', 500)

('T102', 12000)

('T103', 800)

('T104', 15000)

('T105', 700)

('T106', 25000)

Risk Analysis:

('T101', 500, 'Low Risk')

('T102', 12000, 'High Risk')

('T103', 800, 'Low Risk')

('T104', 15000, 'High Risk')

('T105', 700, 'Low Risk')

('T106', 25000, 'High Risk')

Performance Summary

High Risk Transactions: 3

Low Risk Transactions: 3

Improvements:

1. Distributed Processing
2. RDD Caching
3. Feature Engineering
4. Risk-Based Decision Making
5. Scalable Architecture